Algorytmy optymalizacji inspirowane naturą
Laboratorium 0

Magdalena Skrobała 263809

Październik 2025

Spis treści

1 Wstęp teoretyczny **3**

2 Opis problemu **3**

3 Opis badań i eksperymentów **3**
3.1 Dane wejściowe . . . 3
3.2 Algorytm losowy . . . 4
3.3 Algorytm zachłanny . . . 4
3.4 Algorytm ewolucyjny . . . 4
3.5 Algorytm Tabu Search . . . 5

4 Analiza wyników i wnioski **5**
4.1 Instancja A-n32-k5 . . . 7
4.2 Instancja A-n37-k6 . . . 8
4.3 Instancja A-n39-k5 . . . 10
4.4 Instancja A-n45-k6 . . . 11
4.5 Instancja A-n48-k7 . . . 13
4.6 Instancja A-n54-k7 . . . 14
4.7 Instancja A-n60-k9 . . . 17
4.8 Optymalizacja najtrudniejszych instancji . . . 18
4.9 Badanie wpływu wartości dla różnych parametrów algorytmu ewolucyjnego . . . 21

5 Podsumowanie **24**

1 Wstęp teoretyczny

Współczesne problemy optymalizacyjne, szczególnie te należące do klasy NP-trudnych, jak problem marszrutyzacji pojazdów (cVRP – Capacitated Vehicle Routing Problem), wymagają stosowania metod, które pozwalają na efektywne przeszukiwanie ogromnych przestrzeni rozwiązań. W takich przypadkach klasyczne algorytmy deterministyczne okazują się niewystarczające lub zbyt kosztowne obliczeniowo. Z tego względu szczególne znaczenie zdobyły metaheurystyki inspirowane procesami naturalnymi, w tym algorytmy ewolucyjne.

Algorytmy ewolucyjne (EA) stanowią rodzinę metod optymalizacji opartych na zasadach doboru naturalnego i ewolucji biologicznej. Nie są to algorytmy – nie gwarantują znalezienia rozwiązania optymalnego – lecz metaheurystyki, których celem jest iteracyjne ulepszanie populacji potencjalnych rozwiązań. Każde rozwiązanie problemu traktowane jest jako osobnik, podlegający ocenie za pomocą zdefiniowanej funkcji celu. Proces ewolucyjny obejmuje operacje selekcji, krzyżowania oraz mutacji, które pozwalają na eksplorację i eksploatację przestrzeni możliwych rozwiązań.

Zastosowanie algorytmów ewolucyjnych do problemu cVRP wymaga odpowiedniej reprezentacji osobnika (np. permutacji miast), zaprojektowania funkcji oceny mierzącej długość trasy oraz zdefiniowania mechanizmów zachowania poprawności rozwiązania (np. respektowania pojemności pojazdów). Problem cVRP polega na znalezieniu zbioru tras rozpoczynających się i kończących w magazynie, tak aby każde miasto zostało odwiedzone dokładnie raz, a łączny koszt przejazdu był minimalny. Ponieważ problem ten jest NP-trudny, tradycyjne metody dokładne są w praktyce nieefektywne, co uzasadnia użycie metaheurystyk.

W algorytmach ewolucyjnych kluczową rolę odgrywa równowaga między eksploracją, a eksploatacją przestrzeni rozwiązań. Mechanizmy selekcji (np. turniej, ruletka) odpowiadają za wybór najlepiej rokujących osobników, krzyżowanie (OX - Ordered Crossover, PMX - Partially Matched Crossover, CX - Cycle Crossover) pozwala na wymianę informacji między rozwiązaniami, a mutacja (np. swap, inwersja) wprowadza różnorodność genetyczną, zapobiegając przedwczesnej zbieżności do minimum lokalnego. Zbyt silne ciśnienie selekcyjne prowadzi do utraty różnorodności, natomiast zbyt słabe – do braku postępu ewolucji.

W celu oceny skuteczności algorytmów ewolucyjnych konieczne jest porównanie ich z metodami nieewolucyjnymi, takimi jak algorytm losowy, algorytm zachłanny. Metoda losowa polega na generowaniu przypadkowych rozwiązań, co umożliwia ocenę jakości przestrzeni rozwiązań, natomiast algorytm zachłanny konstruuje trasę sekwencyjnie, wykorzystując lokalną informację (np. wybór najbliższego miasta).

Celem niniejszego ćwiczenia jest implementacja algorytmu genetycznego oraz porównanie jego efektywności z metodami: losową, zachłanną i jedną inną wybraną metaheurystyką w kontekście problemu cVRP. Badaniu podlegać będzie wpływ parametrów ewolucyjnych (prawdopodobieństwo mutacji i krzyżowania, typ selekcji) na osiągane rezultaty oraz stabilność metody. Wyniki zostaną przeanalizowane na podstawie wielokrotnych uruchomień, przy użyciu statystyk takich jak najlepsza wartość, średnia oraz odchylenie standardowe. Uzyskane obserwacje pozwolą zrozumieć mechanizmy działania metaheurystyk i ich przydatność w rozwiązywaniu złożonych problemów optymalizacyjnych.

2 Opis problemu

Problem Capacitated Vehicle Routing Problem (cVRP), czyli problem wyznaczania tras pojazdów o ograniczonej pojemności, stanowi uogólnienie klasycznego problemu komiwojażera (TSP). Rozwiązaniem problemu jest zaplanowanie tras dla pojazdów w taki sposób, aby każda lokalizacja została obsłużona dokładnie jeden raz, przy jednoczesnym zachowaniu ograniczeń ładowności oraz minimalizacji całkowitego kosztu przejazdu.

Każdy pojazd rozpoczyna trasę w centralnym magazynie (depot), a po zrealizowaniu dostaw do przypisanych punktów musi powrócić do punktu startowego. Rozwiązanie problemu przedstawia się jako sekwencję odwiedzanych klientów, przy czym żaden z nich nie może zostać pominięty ani odwiedzony więcej niż jeden raz. Głównym zadaniem jest zatem znalezienie możliwie najkrótszych tras spełniających wszystkie narzucone ograniczenia.

3 Opis badań i eksperymentów

3.1 Dane wejściowe

Do wykonania ćwiczenia zostało wykorzystanych siedem różnych plików wejściowych. Podzielono je na instancje o poziomie podstawowym (A-n32-k5, A-n37-k6, A-n39-k5, A-n45-k6, A-n48-k7) oraz instancje o poziomie trudnym (A-n54-k7, A-n60-k9). Każdy plik reprezentuje instancję problemu trasowania pojazdów o ograniczonej pojemności (cVRP), w której należy zoptymalizować trasy dostaw przy minimalnym koszcie całkowitym.

Instancje zawierają liczbę lokacji (w tym magazyn), wraz z wartością zapotrzebowania, liczbę oraz maksymalną pojemność pojazdów. Odległości między punktami są obliczane na podstawie współrzędnych euklidesowych. Wykorzystanie różnych instancji pozwala na ocenę skuteczności implementowanego algorytmu w wa-

runkach o różnym stopniu trudności. Porównanie wyników dla instancji podstawowych i trudnych umożliwia analizę wpływu liczby lokacji, pojemności pojazdów oraz rozkładu zapotrzebowania na jakość rozwiązania.

3.2 Algorytm losowy

Algorytm losowy stanowi najprostszą metodę generowania rozwiązań dla problemu marszrutyzacji pojazdów (cVRP). Tworzy on losową trasę spełniającą ograniczenia pojemnościowe. Choć nie zapewnia rozwiązania optymalnego, stanowi punkt odniesienia do porównań z bardziej zaawansowanymi metodami optymalizacji.

Opis działania algorytmu Pobierane są dane wejściowe zawierające listę miast, macierz odległości oraz maksymalną pojemność pojazdu. Lista miast (z wyłączeniem magazynu) jest losowo mieszana, co zapewnia różnorodność tras. Algorytm stopniowo dodaje miasta do trasy, sprawdzając, czy pojazd ma wystarczającą pojemność. w przypadku jej przekroczenia pojazd wraca do magazynu i rozpoczyna nową trasę. Całkowity koszt to suma odległości między kolejnymi miastami. Po odwiedzeniu wszystkich miast pojazd wraca do magazynu.

Algorytm losowy pozwala na szybkie wygenerowanie poprawnej trasy, lecz nie gwarantuje jakości rozwiązania.

Parametry wejściowe

- Ilość iteracji: 10000

3.3 Algorytm zachłanny

Algorytm zachłanny jest klasyczną heurystyką stosowaną w problemach optymalizacyjnych, takich jak cVRP. Jego działanie oparte jest o podejmowanie lokalnie optymalnych decyzji na każdym etapie.

Opis działania algorytmu Rozpoczęcie trasy z magazynu. Spośród nieodwiedzonych lokacji wybierane jest miasto najbliższe aktualnej pozycji pojazdu, które może zostać obsłużone bez przekroczenia pojemności. Miasta są dodawane do trasy do momentu wyczerpania pojemności, po czym pojazd wraca do magazynu i rozpoczyna nową trasę. Koszt to suma odległości między kolejnymi punktami trasy. Po odwiedzeniu wszystkich miast pojazd wraca do magazynu. Trasa rozpoczyna się zawsze w innym mieście, aż algorytm wykona się tyle razy, ile miast jest w danej instancji problemu.

Parametry wejściowe

- Ilość iteracji: Równa liczbie miast

3.4 Algorytm ewolucyjny

Algorytm ewolucyjny (AE) jest metaheurystyką inspirowaną procesem ewolucji biologicznej. Wykorzystuje operatory takie jak selekcja, krzyżowanie i mutacja, aby iteracyjnie ulepszać populację możliwych rozwiązań.

Opis działania algorytmu

1. **Inicjalizacja populacji** – Osobniki są generowane losowo jako permutacje miast. Sprawdzana jest poprawność względem pojemności pojazdów.
2. **Selekcja turniejowa** – Spośród losowo wybranych osobników wybieranych jest x najlepszych, które przechodzą do kolejnej generacji.
3. **Krzyżowanie (OX)** – Fragment jednej trasy jest kopiowany, a pozostałe miasta są uzupełniane na podstawie kolejności w drugim rodzicu.
4. **Mutacja** – Zastosowana została mutacja typu inwersja.
5. **Ewolucja wielopokoleniowa** – Proces trwa aż do spełnienia warunku zakończenia.
6. **Ocena** – Po każdej generacji oceniane są najlepsze, średnie i najgorsze rozwiązania.

Parametry wejściowe

- Rozmiar populacji: 100
- Ilość generacji: 100

- Prawdopodobieństwo krzyżowania: 0.7
- Prawdopodobieństwo mutacji: 0.4
- Rozmiar turnieju: 5
- Rozmiar elity: 1

3.5 Algorytm Tabu Search

Algorytm Tabu Search (TS) jest metaheurystyką wykorzystującą pamięć przeszłych rozwiązań, w postaci listy tabu, w celu unikania powrotu do wcześniej analizowanych obszarów przestrzeni rozwiązań.

Opis działania algorytmu

- **Inicjalizacja** – Generowanie początkowej losowej trasy.
- **Poszukiwanie sąsiadów** – Modyfikacja rozwiązania poprzez zamianę pozycji dwóch miast.
- **Lista tabu** – Zapamiętywanie zabronionych ruchów na określoną liczbę iteracji.
- **Ocena** – Wybór najlepszego rozwiązania nieobjętego tabu.

Parametry wejściowe

- Liczba iteracji: 100
- Rozmiar listy tabu: 10
- Operator ruchu: zamiana dwóch miast (swap)
- Aspiracja: dopuszczenie ruchu tabu, jeśli poprawia najlepsze globalne rozwiązanie

4 Analiza wyników i wnioski

Tabela 1 przedstawia porównanie skuteczności czterech metod optymalizacji: algorytmu losowego, algorytmu zachłannego, algorytmu ewolucyjnego oraz metody Tabu Search, zastosowanych do siedmiu instancji problemu cVRP. Dla każdej instancji podano wynik optymalny oraz: najlepszy wynik (best), najgorszy wynik (worst), średnią (avg) oraz odchylenie standardowe (std), obliczone na podstawie wielokrotnych uruchomień (10 000 dla algorytmu losowego, n = liczbie miast dla algorytmu zachłannego oraz 10-krotnie dla pozostałych metod).

W przypadku algorytmu losowego wartości średnie są znacznie oddalone od wyniku optymalnego, co potwierdza niską skuteczność tej metody oraz brak mechanizmów eksploracyjno-eksploatacyjnych. Duże odchylenia standardowe wskazują na wysoką zmienność rezultatów i brak stabilności.

Tabela 1: Porównanie wyników metod optymalizacji dla instancji cVRP.

instancja	Opt.	Alg.losowy [10k]				Alg.zachłanny [n]				Alg.Ewolucyjny [10x]				TS [10x]		
	wynik	**best**	worst	avg	std	**best**	worst	avg	std	**best**	worst	avg	std	**best**	worst	avg
A-n32-k5	784	**1566,95**	2553,64	2120,33	140,245	**1030,66**	1250,32	1150,84	61,6481	**820,598**	1002,84	906,356	57,7969	**845,056**	1015,54	946,725
A-n37-k6	949	**1683,88**	2654,51	2217,69	129,773	**1177,91**	1467,05	1325,09	58,977	**1068,71**	1201,30	1109,91	42,8465	**1018,28**	1141,09	1070,54
A-n39-k5	822	**1690,28**	2603,29	2181,55	130,141	**998,946**	1262,78	1135,91	60,03	**916,44**	1122,85	1030,26	56,3199	**906,798**	991,29	953,531
A-n45-k6	944	**2135,48**	3368,51	2814,76	152,927	**1212,47**	1727,75	1400,28	137,945	**1178,05**	1378,99	1295,83	64,1375	**1117,18**	1318,77	1214,04
A-n48-k7	1073	**2306,72**	3512,42	2942,19	169,53	**1380,80**	1695,91	1473,78	72,7758	**1318,15**	1492,58	1403,47	56,9478	**1188,38**	1415,94	1302
A-n54-k7	1167	**2418,32**	3921,86	3295,84	172,332	**1360,48**	1621,82	1524,48	62,7286	**1510,48**	1756,44	1657,07	67,8453	**1274,67**	1527,44	1444,91
A-n60-k9	1354	**2896,67**	4371,14	3701,29	179,069	**1686,52**	1972,17	1852,75	68,7713	**1774,39**	2076,03	1937,18	92,1196	**1601,63**	1809,56	1696,55

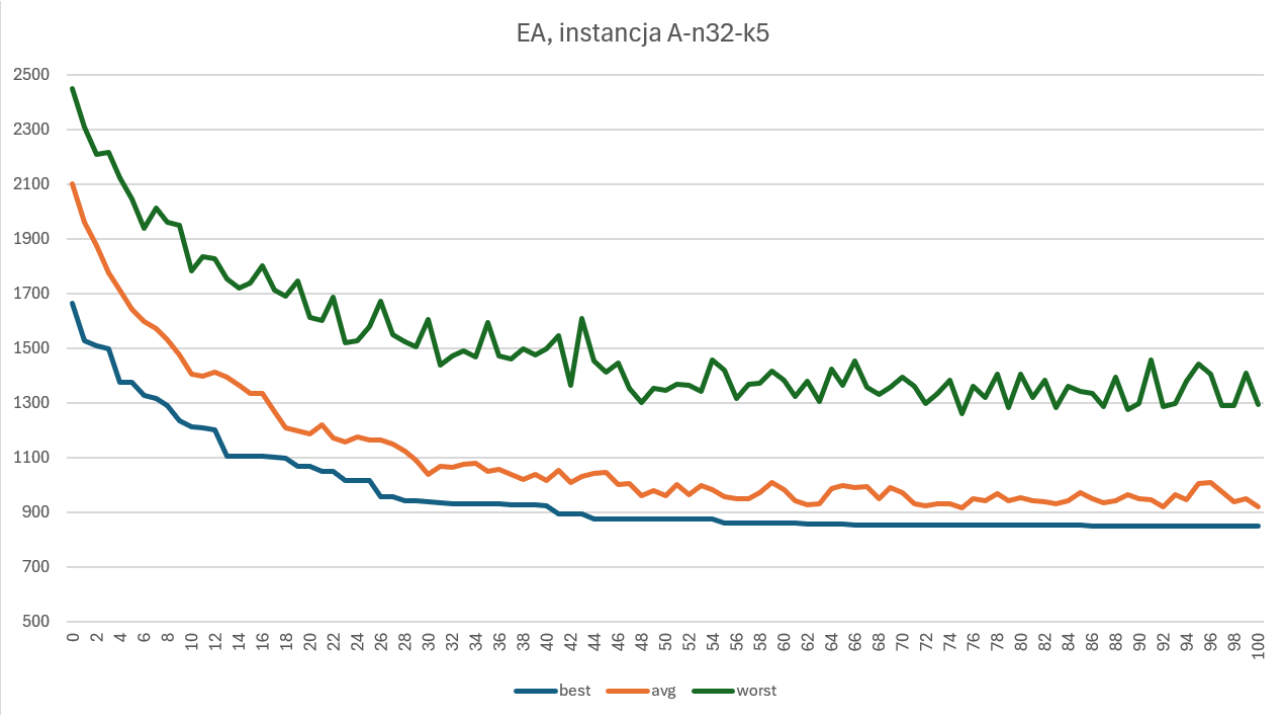
Algorytm zachłanny osiąga lepsze wyniki niż metoda losowa, szczególnie pod względem wartości best oraz avg. Zachowanie lokalnej optymalności pozwala mu na generowanie spójnych i powtarzalnych trajektorii, ale pozostaje podatny na utykanie w minimach lokalnych, co obserwowane jest szczególnie w trudniejszych instancjach (np. A-n54-k7, A-n60-k9).

Wysokie wyniki uzyskano w przypadku algorytmu ewolucyjnego, który w większości instancji osiąga wartości bliskie optimum. Niewielkie odchylenie standardowe świadczy o stabilności metody oraz zdolności do skutecznej eksploatacji przestrzeni rozwiązań. Ewolucja populacji poprzez krzyżowanie i mutacje pozwala na odkrywanie lepszych tras niż w przypadku metod zachłannych i czysto losowych.

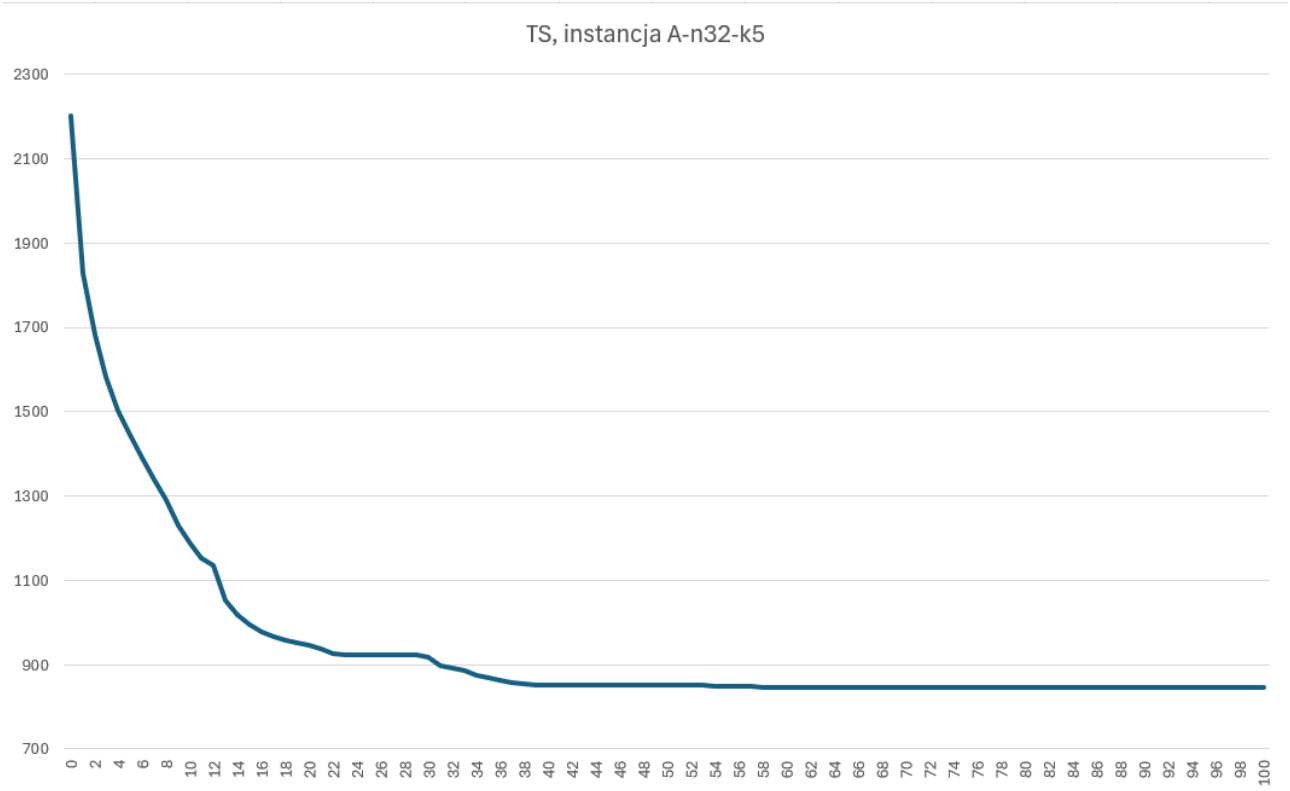
Tabu Search uzyskał najlepsze wyniki jakościowe, często zbliżone do algorytmu ewolucyjnego i optimum. w instancjach trudnych TS osiąga niższe worst i avg w porównaniu do heurystyki zachłannej.

4.1 Instancja A-n32-k5

Dla instancji A-n32-k5, której optymalny wynik to 784, najlepsze rezultaty uzyskały algorytmy metaheurystyczne. Algorytm ewolucyjny osiągnął najkorzystniejszy wynik 820 oraz średnią 906, co świadczy o wysokiej skuteczności jego operatorów selekcji i mutacji w prowadzeniu populacji w kierunku rozwiązań bliskich optimum. Algorytm Tabu Search znalazł rozwiązanie o koszcie 845 i utrzymał średnią na poziomie 946, wykazując jednocześnie dobrą stabilność przeszukiwania. Algorytm zachłanny zatrzymał się na 1030, co wskazuje na szybkie uwięzienie w lokalnym optimum. Najsłabsze wyniki osiągnął algorytm losowy, którego najlepszy rezultat wyniósł 1566, a średnia przekroczyła 2500. Potwierdza to brak efektywnego mechanizmu sterowania procesem poszukiwania w algorytmie losowym. Przebieg algorytmu ewolucyjnego ujawnia szybki spadek kosztu w początkowych pokoleniach oraz stabilizację w końcowej fazie, przy zachowaniu zmienności najgorszych osobników. Taki kształt wykresu potwierdza zdolność do eksploracji przestrzeni rozwiązań.



Rysunek 1: Wykres algorytmu ewolucyjnego dla instancji A-n32-k5.



Rysunek 2: Wykres Tabu Search dla instancji A-n32-k5.

EA – najlepsze rozwiązanie

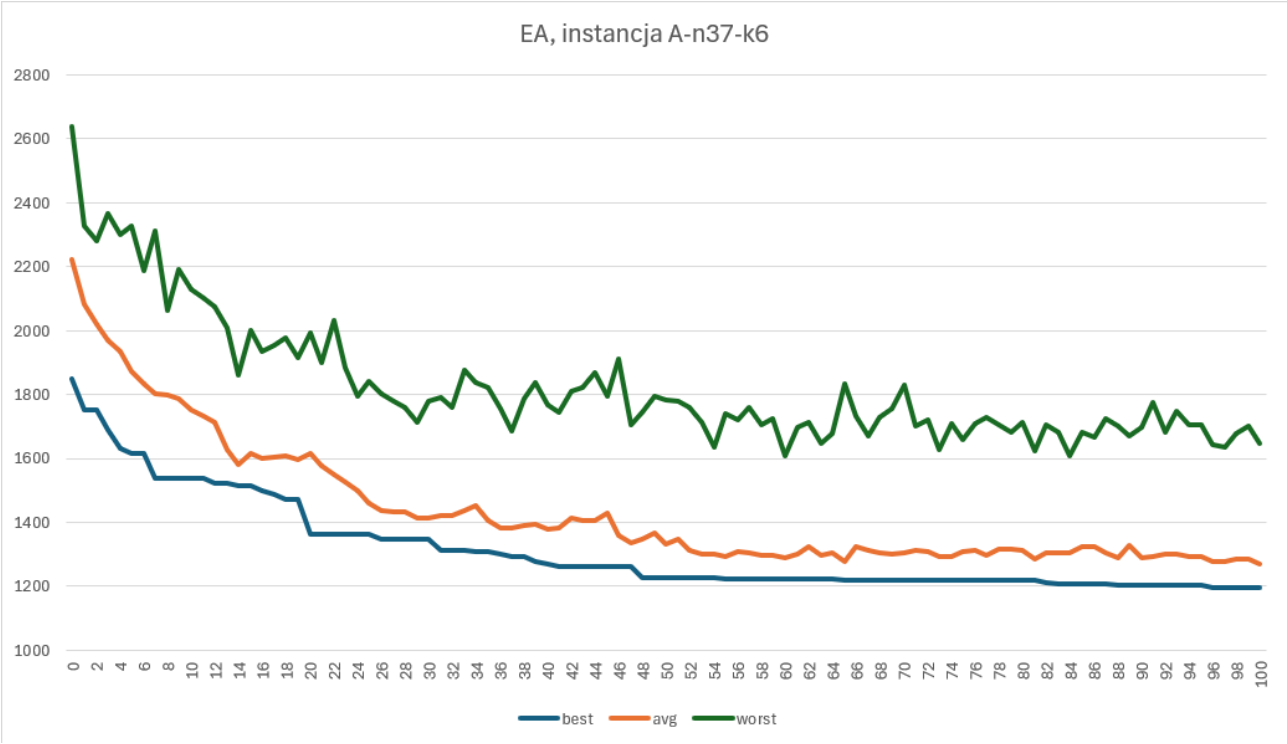
- **Route 1:** 14, 18, 20, 32, 22, 17, 27
- **Route 2:** 19, 9, 10, 23, 16, 30, 11, 26, 6, 21
- **Route 3:** 7, 4, 3, 24, 5, 12, 29, 15
- **Route 4:** 25, 8, 2, 13, 31
- **Route 5:** 28

TS – najlepsze rozwiązanie

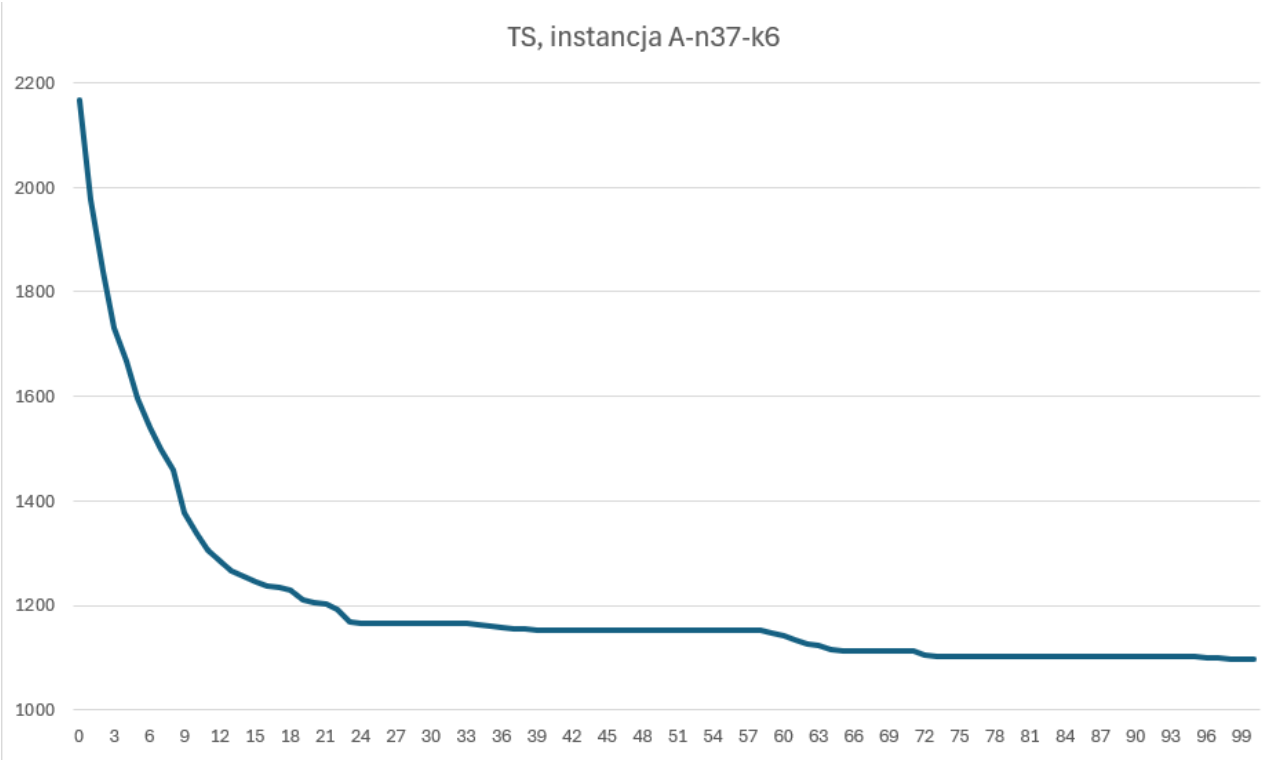
- **Route 1:** 2, 22, 32, 20, 18, 14
- **Route 2:** 4, 3, 24, 29, 5, 12, 9, 19, 30, 21
- **Route 3:** 23, 10, 16, 11, 26, 6
- **Route 4:** 28, 25, 15, 7, 8, 13
- **Route 5:** 31, 17, 27

4.2 Instancja A-n37-k6

W przypadku instancji A-n37-k6 o optymalnej wartości 949, algorytm ewolucyjny ponownie wykazał się wysoką skutecznością, osiągając najlepszy wynik 1068 oraz średnią 1109, co oznacza zbliżenie do optimum mimo zwiększonej złożoności problemu. Tabu Search uzyskał najlepsze rezultaty spośród wszystkich metod, poprawiając wynik algorytmu ewolucyjnego o 50, a zatem uzyskując best równe 1018 i średnią 1070. Algorytm zachłanny, z wynikiem 1177 i średnią 1325, dostarczył stabilnych, ale wyraźnie gorszych rezultatów, wskazując na brak adaptacji do struktury problemu. Algorytm losowy ponownie okazał się najmniej efektywny, osiągając średnią ponad 2200, co potwierdza jego przypadkowy charakter. Analiza przebiegu algorytmu ewolucyjnego ujawnia gwałtowne obniżanie kosztu w początkowej fazie działania oraz stopniowe wygładzanie krzywej średniej. Podobny charakter wykresu uzyskano dla każdej z badanych instancji.



Rysunek 3: Wykres algorytmu ewolucyjnego dla instancji A-n37-k6.



Rysunek 4: Wykres Tabu Search dla instancji A-n37-k6.

EA – najlepsze rozwiązanie

- **Route 1:** 27, 11, 31, 16, 26
- **Route 2:** 15, 35, 32, 14, 17
- **Route 3:** 28, 33, 12, 13, 23, 24, 29, 3, 5
- **Route 4:** 20, 34, 10, 22, 18, 19, 8

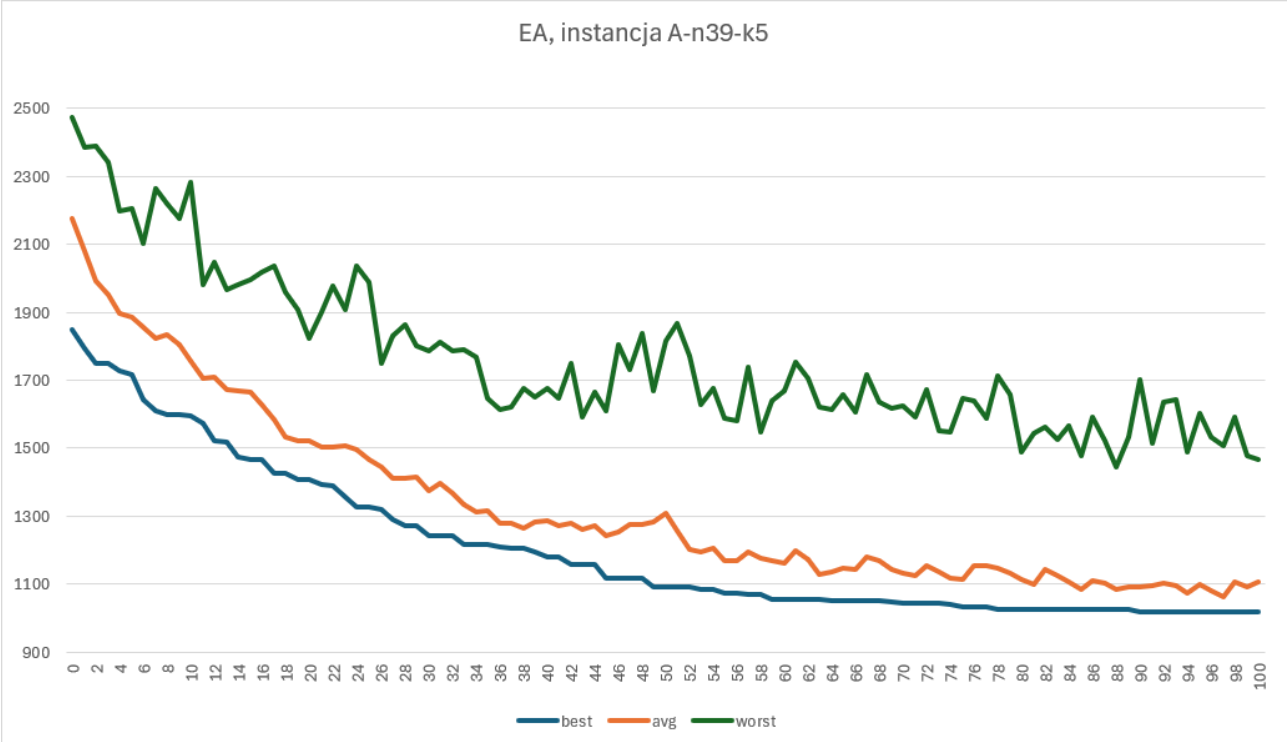
- **Route 5:** 7, 9, 4, 6, 2, 21
- **Route 6:** 36, 25
- **Route 7:** 30, 37

TS – najlepsze rozwiązanie

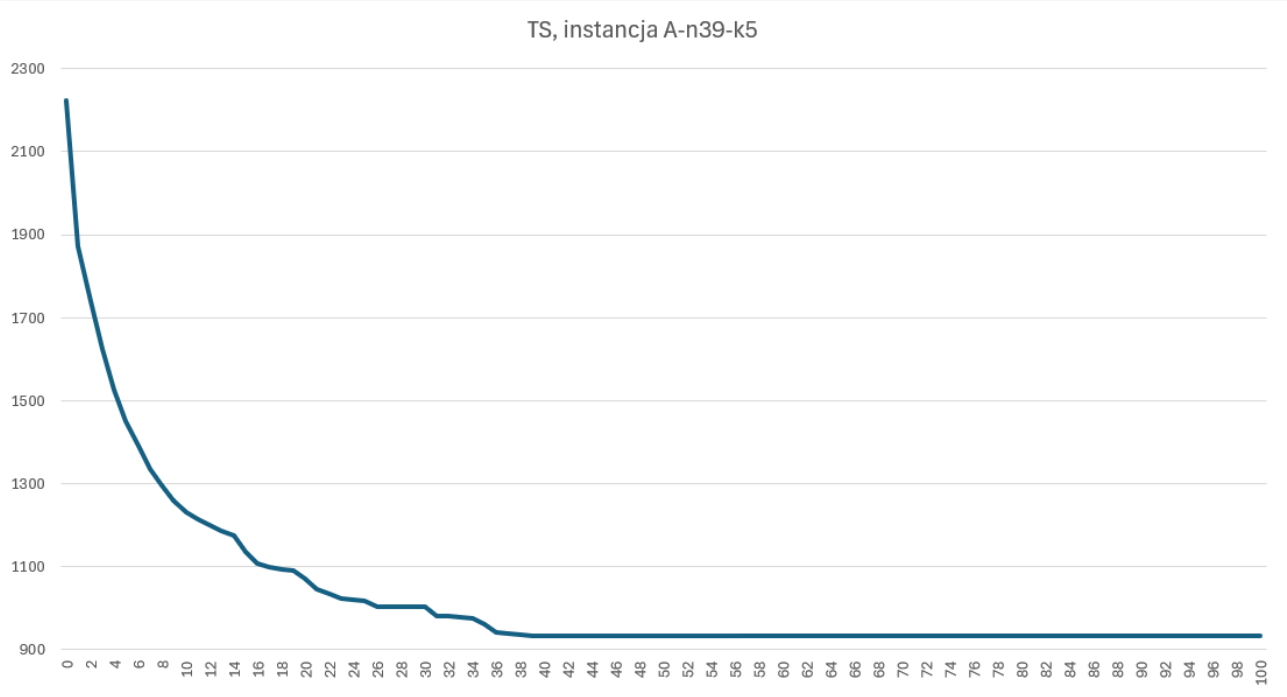
- **Route 1:** 18, 35, 32, 37, 30
- **Route 2:** 14, 12, 13, 22, 10, 20, 19
- **Route 3:** 15, 7, 9, 6, 4, 2, 21
- **Route 4:** 17, 33, 16, 31, 8
- **Route 5:** 25, 28, 36, 26
- **Route 6:** 34, 3, 29, 24, 23, 11, 27, 5

4.3 Instancja A-n39-k5

Dla instancji A-n39-k5, gdzie wartość optymalna wynosi 822, Tabu Search uzyskał najlepszą średnią jakość rozwiązań, osiągając 954 przy najlepszym wyniku 907. Algorytm ewolucyjny wygenerował rozwiązania nieco słabsze – najlepszy koszt 916 i średnia 1030 – lecz różnica pomiędzy najlepszym wynikiem EA i TS wynosi jedynie 10(!). Algorytm zachłanny osiągnął wynik 998 i średnią 1135, co plasuje go wyraźnie poniżej algorytmów metaheurystycznych. Algorytm losowy zanotował zdecydowanie najgorsze rezultaty, ze średnią ponad 2180, co wskazuje na całkowity brak mechanizmu poprawy rozwiązań. Przebieg ewolucji pokazuje wyraźne tempo poprawy najlepszego osobnika oraz dużą zmienność wartości najgorszych rozwiązań. Świadczy to o zachowaniu różnorodności i unikaniu przedwczesnej konwergencji.



Rysunek 5: Wykres algorytmu ewolucyjnego dla instancji A-n39-k5.



Rysunek 6: Wykres Tabu Search dla instancji A-n39-k5.

EA – najlepsze rozwiązanie

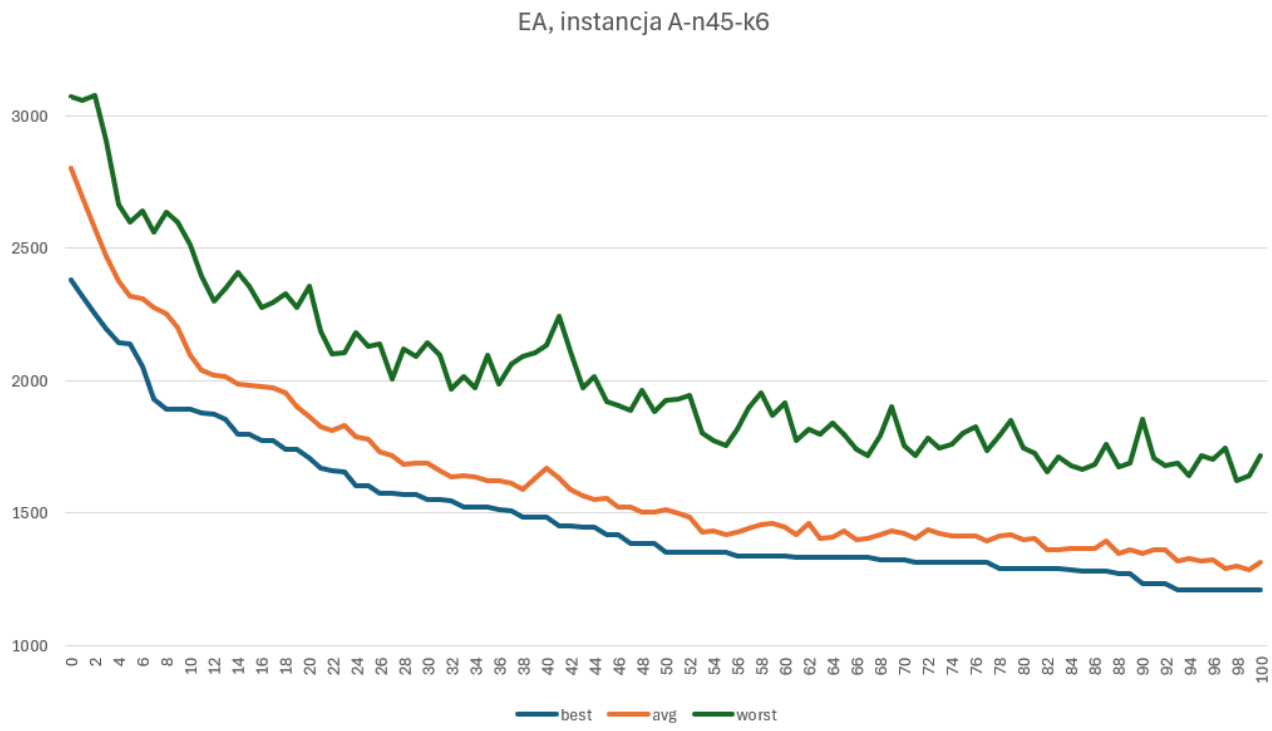
- **Route 1:** 18, 25, 36, 38, 35, 27, 7
- **Route 2:** 34, 30, 8, 4, 23, 33, 3, 12
- **Route 3:** 13, 32, 39, 16, 10, 19, 5
- **Route 4:** 2, 11, 17, 6, 21, 24, 26, 20
- **Route 5:** 15, 22, 31, 14, 29, 37, 28, 9

TS – najlepsze rozwiązanie

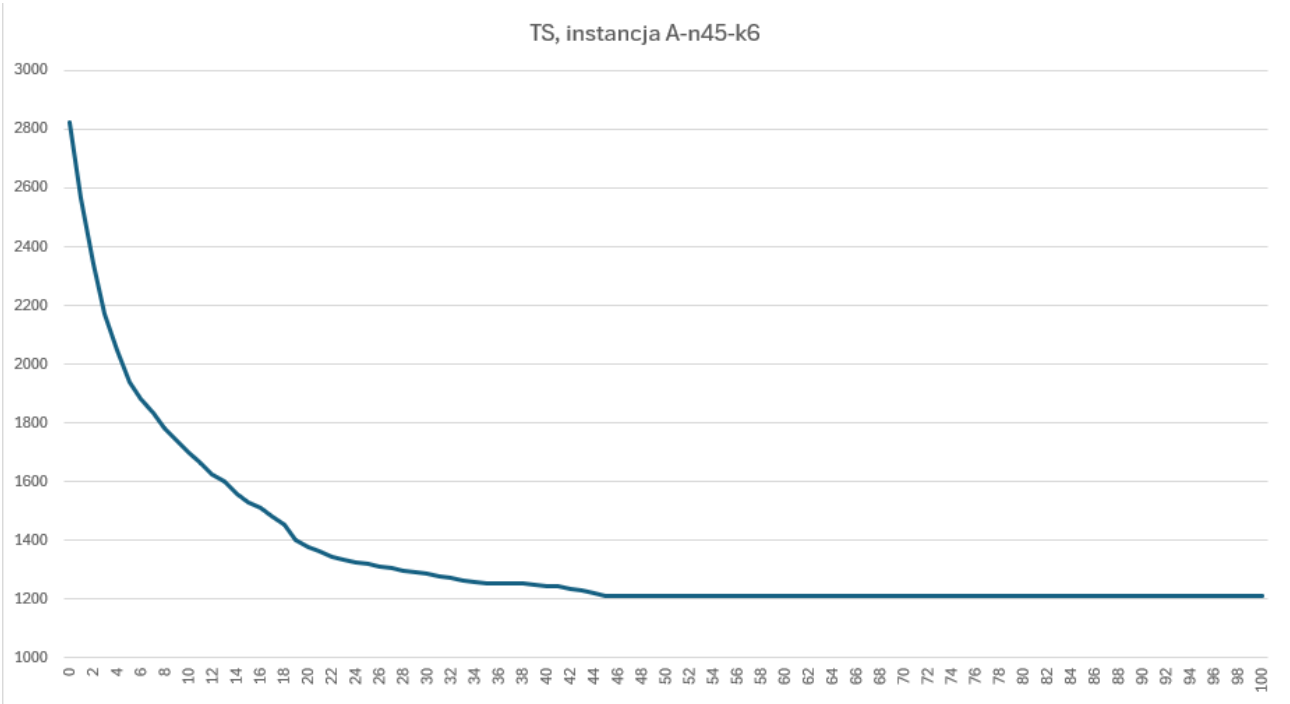
- **Route 1:** 3, 23, 4, 8, 33, 11, 2
- **Route 2:** 12, 18, 25, 36, 38, 29, 14, 31, 22
- **Route 3:** 39, 16, 17, 6, 30, 21, 24, 26, 20
- **Route 4:** 7, 37, 28, 35, 27, 9
- **Route 5:** 5, 19, 10, 13, 32, 34
- **Route 6:** 15

4.4 Instancja A-n45-k6

W instancji A-n45-k6, dla której wartość optymalna wynosi 944, najlepsze wyniki ponownie osiągnęły metody metaheurystyczne, przy czym Tabu Search uzyskał najniższy koszt 1117 oraz średnią 1214, wyraźnie przewyższając algorytm ewolucyjny, który zatrzymał się na poziomie 1178 dla najlepszego rozwiązania i średniej 1295. Wskazuje to, że na tym etapie złożoności problemu TS skuteczniej omija lokalne minima i efektywniej eksploruje przestrzeń rozwiązań. Algorytm zachłanny, pomimo deterministycznego charakteru, uzyskał wynik 1212 i średnią 1400. Najsłabsze rezultaty ponownie odnotował algorytm losowy, którego średnia przekroczyła 2810, a najwyższy koszt sięgał ponad 3300. Przebieg algorytmu ewolucyjnego dla tej instancji pokazał wolniejsze tempo konwergencji w porównaniu do mniejszych instancji.



Rysunek 7: Wykres algorytmu ewolucyjnego dla instancji A-n45-k6.



Rysunek 8: Wykres Tabu Search dla instancji A-n45-k6.

EA – najlepsze rozwiązanie

- **Route 1:** 43, 17, 5, 37, 40, 13, 3, 16, 10
- **Route 2:** 30, 28, 20, 31, 41, 7
- **Route 3:** 2, 45, 25, 36, 15
- **Route 4:** 29, 8, 21, 11, 4, 23
- **Route 5:** 38, 35, 12, 18, 19, 27, 24

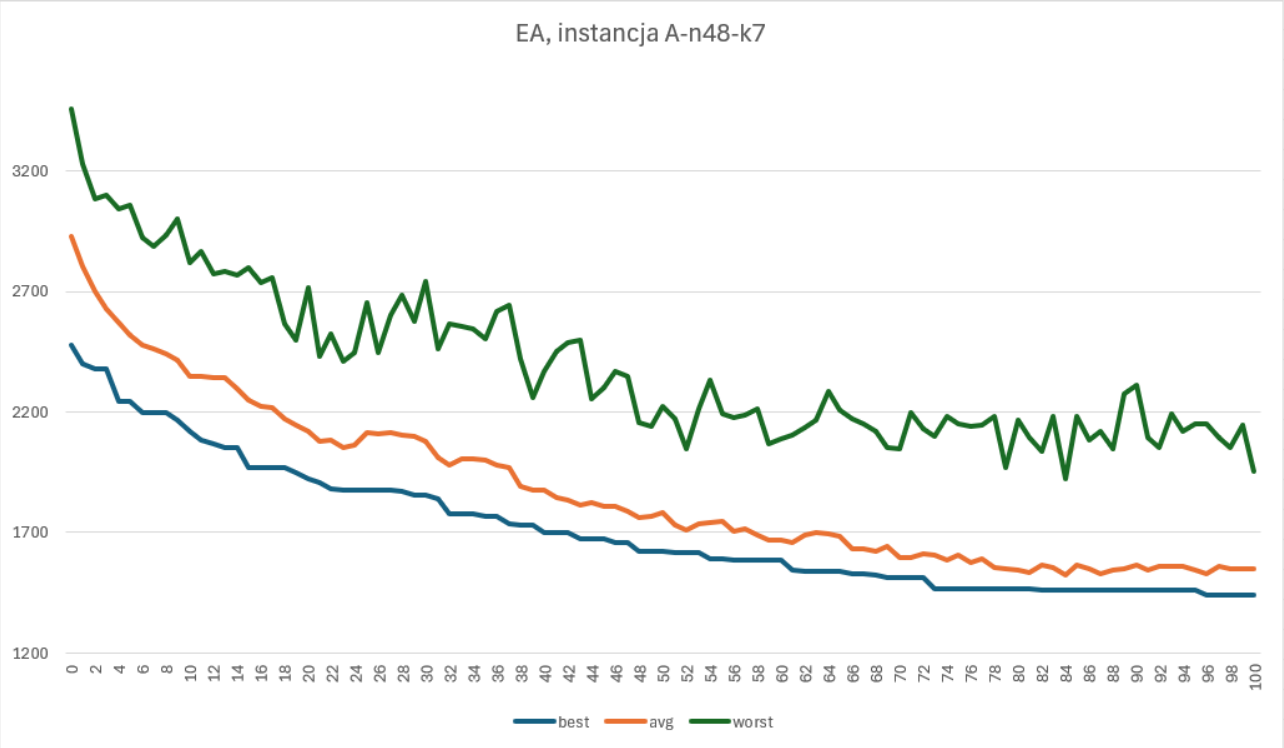
- **Route 6:** 14, 9, 34, 42, 22, 6, 44, 33
- **Route 7:** 26, 39, 32

TS – najlepsze rozwiązanie

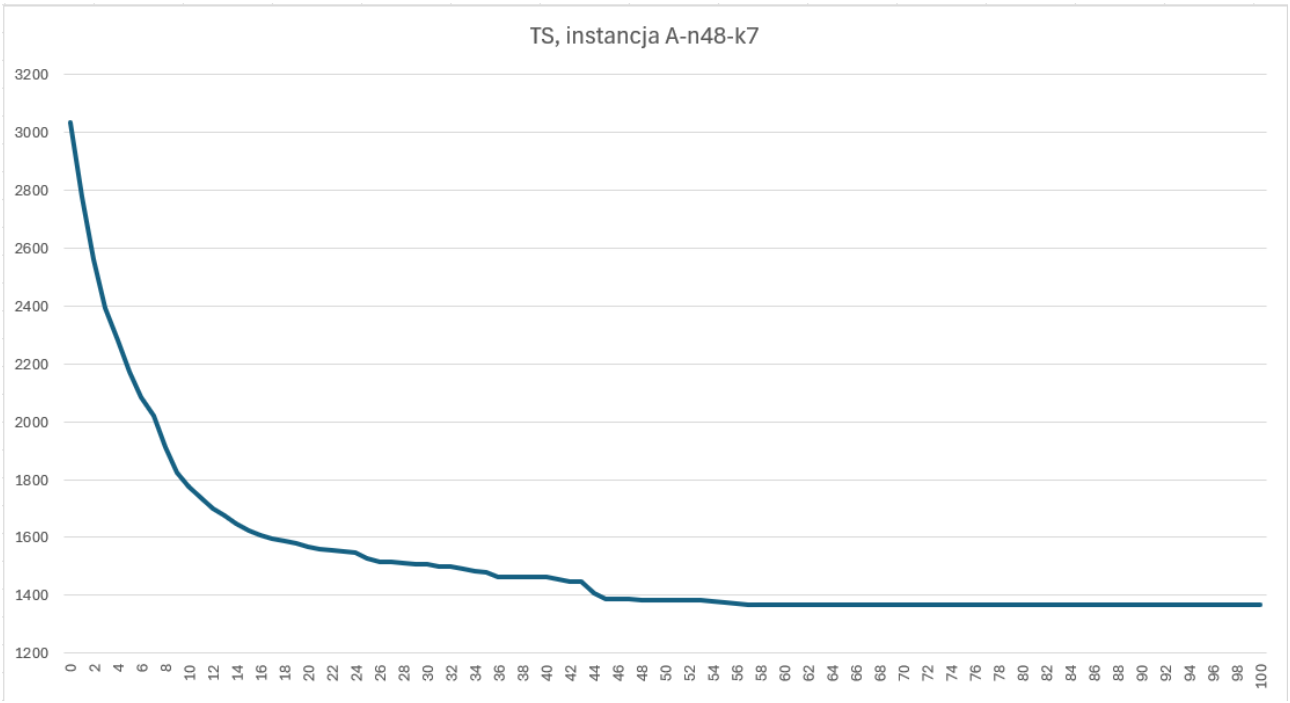
- **Route 1:** 30, 14, 8, 29, 24
- **Route 2:** 23, 11, 17, 5, 43, 37, 40
- **Route 3:** 10, 16, 13, 26, 3, 39, 32, 36
- **Route 4:** 44, 18, 19, 27, 2, 15
- **Route 5:** 45, 7, 25, 38, 35, 28
- **Route 6:** 33, 21, 6, 22, 34, 42, 9, 4
- **Route 7:** 41, 31, 20, 12

4.5 Instancja A-n48-k7

W przypadku instancji A-n48-k7, dla której optymalny koszt wynosi 1073, zauważalne jest dalsze pogłębianie różnic między metodami prostymi a metaheurystykami. Dodatkowo następuje coraz większa rozbieżność pomiędzy wynikami osiąganymi przez Tabu Search i algorytm ewolucyjny. TS uzyskał najlepszy koszt 1188 ze średnią 1302, co czyni go najbardziej efektywną metodą w tej instancji. Algorytm ewolucyjny osiągnął wynik 1318 dla najlepszego rozwiązania i średnią 1403. w związku z tym, nasuwa się wniosek, że przy rosnącej liczbie miast EA zaczyna tracić efektywność, prawdopodobnie z powodu niedostatecznej liczby pokoleń lub nieoptymalnej parametryzacji. Algorytm zachłanny, z wynikiem 1380 i średnią 1473, utrzymuje poprawny, lecz ograniczony poziom jakości. Algorytm losowy, jak w poprzednich przypadkach, dostarczał rozwiązań znacznie oddalonych od optimum, ze średnią ponad 2940 i bardzo dużym rozrzutem wyników.



Rysunek 9: Wykres algorytmu ewolucyjnego dla instancji A-n48-k7.



Rysunek 10: Wykres Tabu Search dla instancji A-n48-k7.

EA – najlepsze rozwiązanie

- **Route 1:** 19, 24, 13, 6, 32, 44
- **Route 2:** 42, 3, 11, 48, 17
- **Route 3:** 34, 31, 47, 14, 43, 12, 10, 28
- **Route 4:** 18, 22, 30, 35, 46, 38, 37, 45
- **Route 5:** 41, 16, 9, 40, 21, 4, 33, 36
- **Route 6:** 29, 25, 5, 27, 8, 20, 26, 39, 23, 15
- **Route 7:** 7, 2

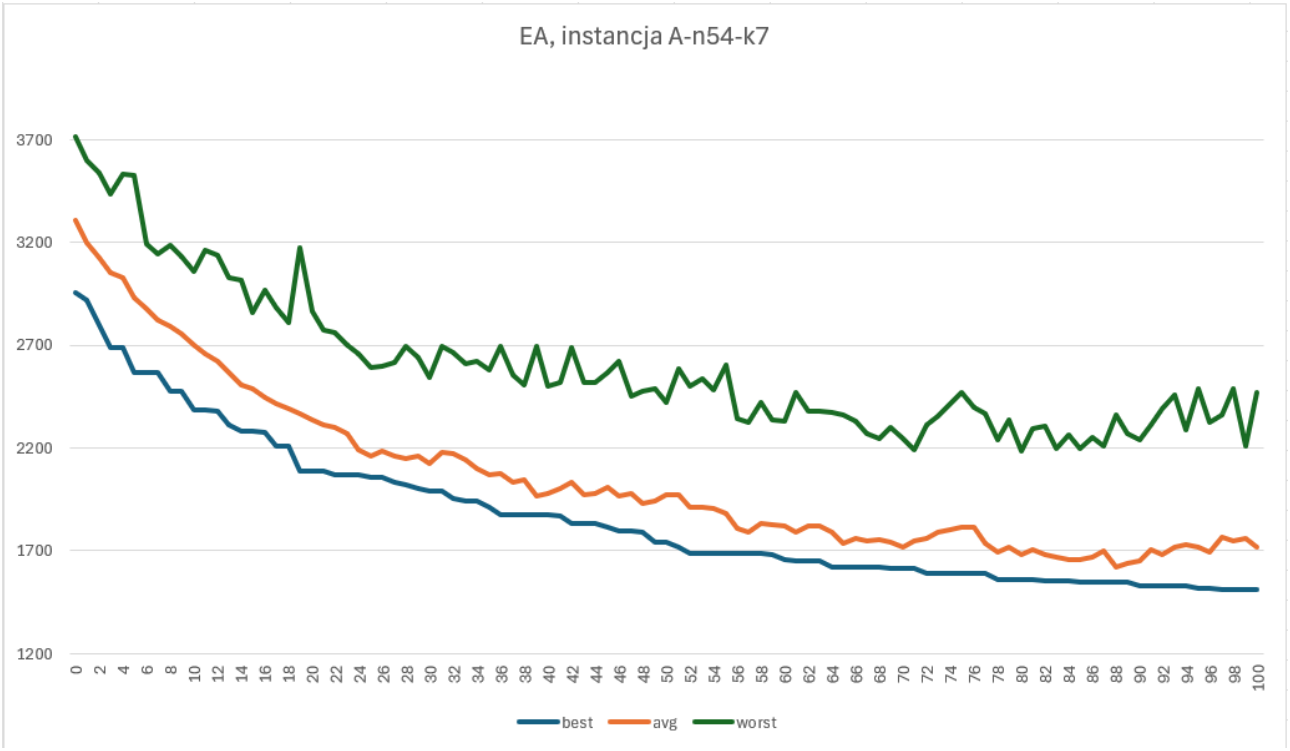
TS – najlepsze rozwiązanie

- **Route 1:** 36, 10, 25, 5, 14, 47, 31, 22, 30, 29
- **Route 2:** 41, 46, 28, 9, 16, 12, 43, 35
- **Route 3:** 23, 7, 2, 6, 32, 13
- **Route 4:** 40, 27, 21, 8, 37, 45, 19
- **Route 5:** 17, 48, 11, 34, 3
- **Route 6:** 33, 39, 20, 38, 4, 26, 44, 24
- **Route 7:** 15, 18, 42

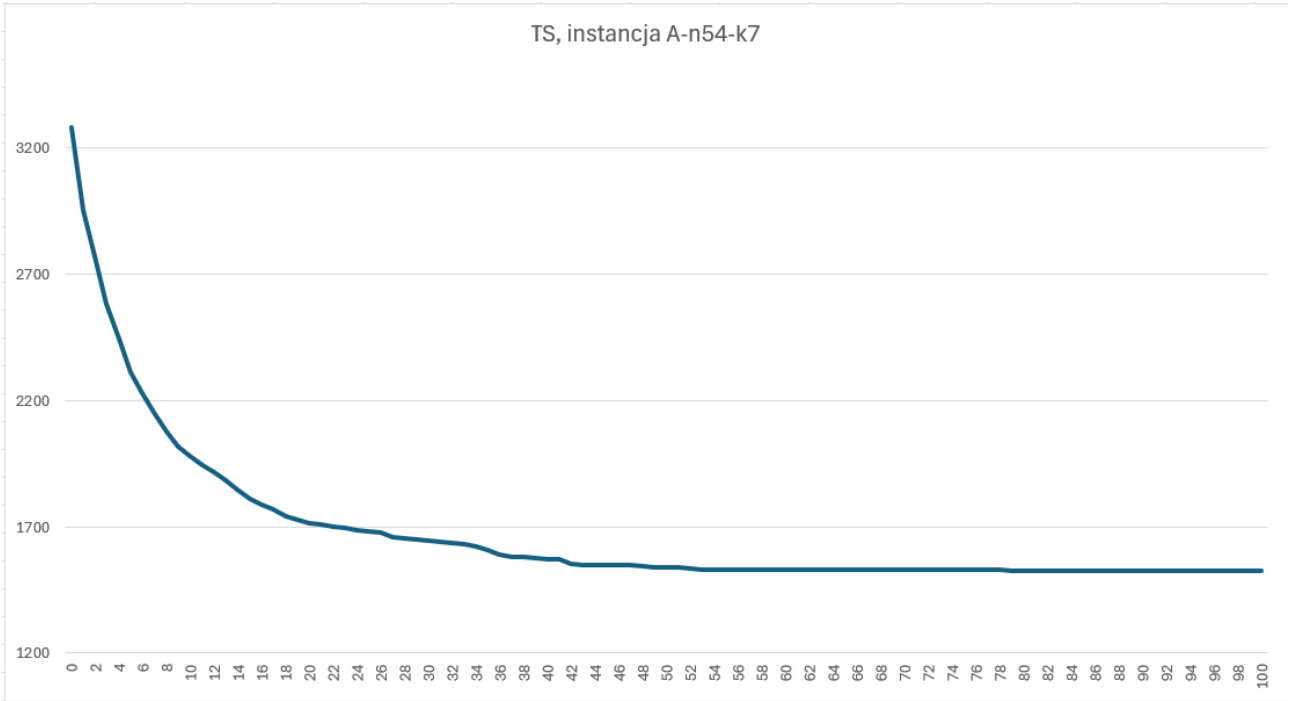
4.6 Instancja A-n54-k7

Instancja A-n54-k7 to pierwsza z dwóch trudnych instancji badanych w ramach tego sprawozdania. Jej optymalny koszt wynosi 1167. Wyraźnie widoczna jest przewaga metody Tabu Search, która osiągnęła najlepszy wynik 1274 i średnią 1444. Jest to wynik mocno zbliżony do optymalnego. Algorytm ewolucyjny, w przeciwieństwie do mniejszych instancji, uzyskał gorsze rezultaty – najlepszy koszt 1510 i średnią 1657 – co wskazuje na osłabioną skuteczność operatorów genetycznych przy zwiększonej złożoności problemu. Zdecydowanie powinno się zwiększyć wielkość populacji, bądź liczbę pokoleń, w celu uzyskania lepszych wyników. Algorytm zachłanny,

mimo prostoty, osiągnął wynik 1360 i średnią 1524, co plasuje go wyżej niż EA. Najgorsze rezultaty ponownie zanotował algorytm losowy, którego średnia przekroczyła 3290.



Rysunek 11: Wykres algorytmu ewolucyjnego dla instancji A-n54-k7.



Rysunek 12: Wykres Tabu Search dla instancji A-n54-k7.

EA – najlepsze rozwiązanie

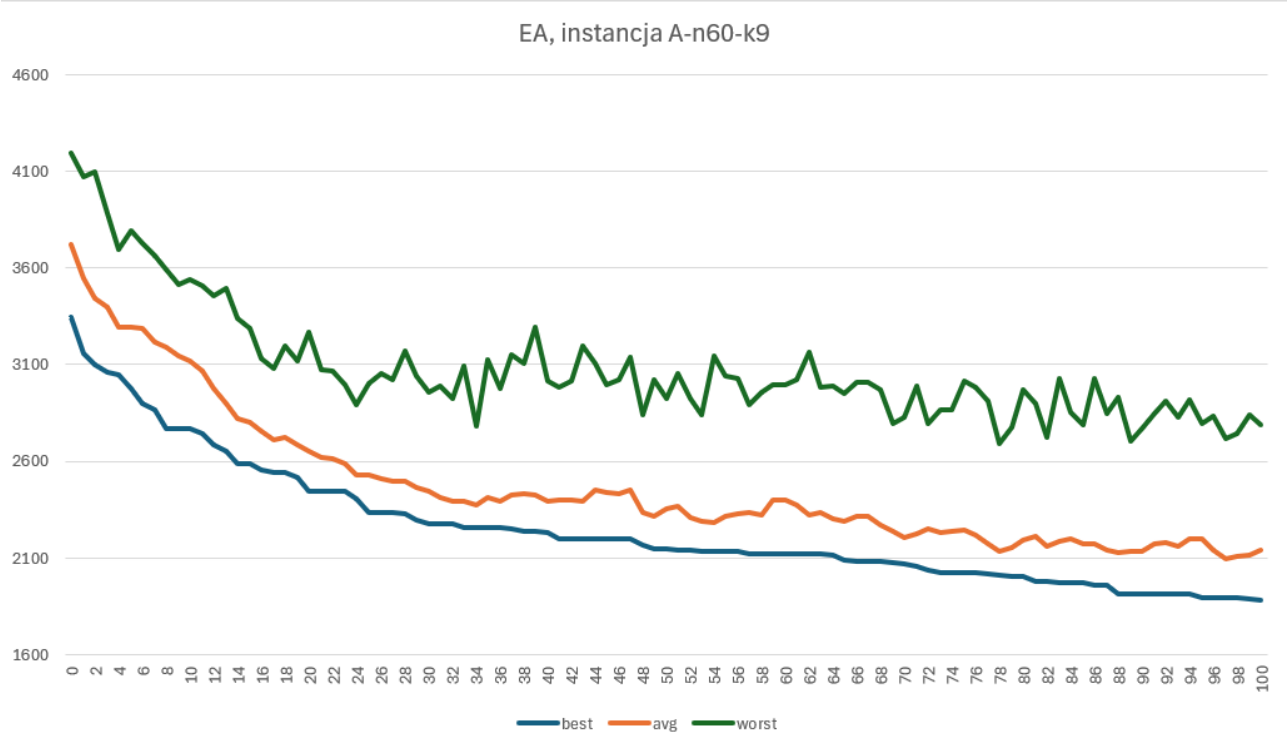
- **Route 1:** 14, 4, 9, 41, 38, 47, 26
- **Route 2:** 44, 29, 6, 8, 37, 51, 40, 19, 5
- **Route 3:** 24, 21, 50, 30, 11, 3, 17, 53
- **Route 4:** 2, 27, 46, 39, 45, 10, 36
- **Route 5:** 23, 32, 20, 49, 15, 25, 33
- **Route 6:** 16, 12, 54, 34, 22, 18
- **Route 7:** 31, 7, 28, 13, 43, 48, 52, 42, 35

TS – najlepsze rozwiązanie

- **Route 1:** 5, 29, 8, 51, 40, 50, 21, 24
- **Route 2:** 44, 36, 37, 2, 18, 7, 33, 53
- **Route 3:** 23, 54, 45, 11, 16
- **Route 4:** 30, 27, 46, 22, 34, 10, 39
- **Route 5:** 26, 48, 52, 25, 43, 47, 42, 35, 31
- **Route 6:** 20, 9, 49, 41, 32, 4, 12, 14
- **Route 7:** 17, 15, 3, 13, 38, 28, 6, 19

4.7 Instancja A-n60-k9

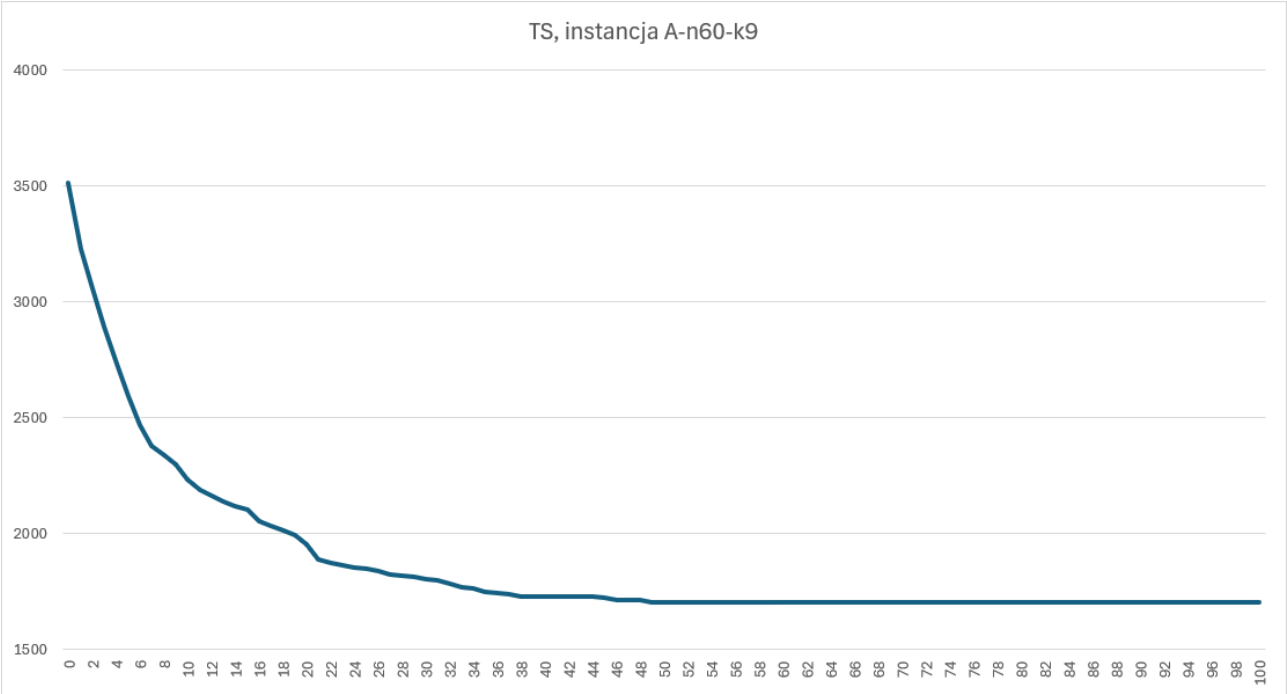
W instancji A-n60-k9, będącej najbardziej złożoną w zestawieniu, z optymalnym wynikiem równym 1354, Tabu Search ponownie okazuje się najbardziej skuteczną metodą, osiągając koszt 1601 i średnią 1696, co wskazuje na najlepszą równowagę między eksploracją a eksploatacją. Algorytm ewolucyjny, z najlepszym wynikiem 1774 i średnią 1937, traci przewagę obserwowaną w mniejszych instancjach. Algorytm zachłanny, z wynikiem 1686 i średnią 1852, mimo swojej deterministyczności, wyprzedza EA pod względem stabilności działania, lecz nie jest w stanie osiągnąć rozwiązań porównywalnych z TS. Najsłabiej wypadł algorytm losowy, którego średnia przekroczyła 3700. Algorytm losowy okazał się całkowicie nieprzydatny w dużych instancjach. Analiza przebiegu ewolucji pokazuje większe "wypłaszczenie"wykresu niż dla jakiejkolwiek z wcześniejszych instancji.



Rysunek 13: Wykres algorytmu ewolucyjnego dla instancji A-n60-k9.

EA – najlepsze rozwiązanie

- **Route 1:** 4, 12, 47, 22, 32, 45, 59, 40
- **Route 2:** 52, 13, 10, 33, 57, 44
- **Route 3:** 29, 49, 37, 54, 41
- **Route 4:** 21, 17, 16, 58, 28, 38, 30, 39, 42
- **Route 5:** 34, 53, 60, 8, 9, 20
- **Route 6:** 7, 3, 23, 43, 55, 2, 50
- **Route 7:** 25, 11, 6, 46
- **Route 8:** 14, 18, 27, 56, 36, 51, 48
- **Route 9:** 26, 24, 35, 31, 5, 15, 19



Rysunek 14: Wykres Tabu Search dla instancji A-n60-k9.

TS – najlepsze rozwiązanie

- **Route 1:** 8, 30, 14, 56, 48
- **Route 2:** 46, 57, 44, 15
- **Route 3:** 35, 3, 37, 11, 49, 25
- **Route 4:** 9, 18, 27, 16, 40, 36, 19
- **Route 5:** 51, 13, 52, 10, 33, 59, 24
- **Route 6:** 26, 7, 22, 5, 12, 41, 4, 21, 17
- **Route 7:** 20, 38, 58, 28, 2, 45, 50, 32, 29, 47
- **Route 8:** 54, 31, 23, 55, 6, 43
- **Route 9:** 42, 53, 60, 39, 34

4.8 Optymalizacja najtrudniejszych instancji

Ze względu na względnie mocno oddalone od optimum wyniki algorytmu genetycznego w dwóch najtrudniejszych instancjach (A-n54-k7 i A-n60-k9), zdecydowano się na powtórzenie eksperymentów z innym zestawem parametrów. Dla algorytmu genetycznego są to:

- Rozmiar populacji: 100
- Ilość generacji: 300
- Prawdopodobieństwo krzyżowania: 0.7
- Rodzaj krzyżowania: OX
- Prawdopodobieństwo mutacji: 0.4
- Rodzaj mutacji: inwersja
- Rozmiar turnieju: 5
- Rozmiar elity: 1

Te zmiany spowodowały również zwiększenie liczby iteracji algorytmu losowego do 30 000.
Pozostałe algorytmy działają bez zmian w stosunku do opisu z rozdziału trzeciego.

Tabela 2: Porównanie wyników metod optymalizacji dla trudnych instancji cVRP.

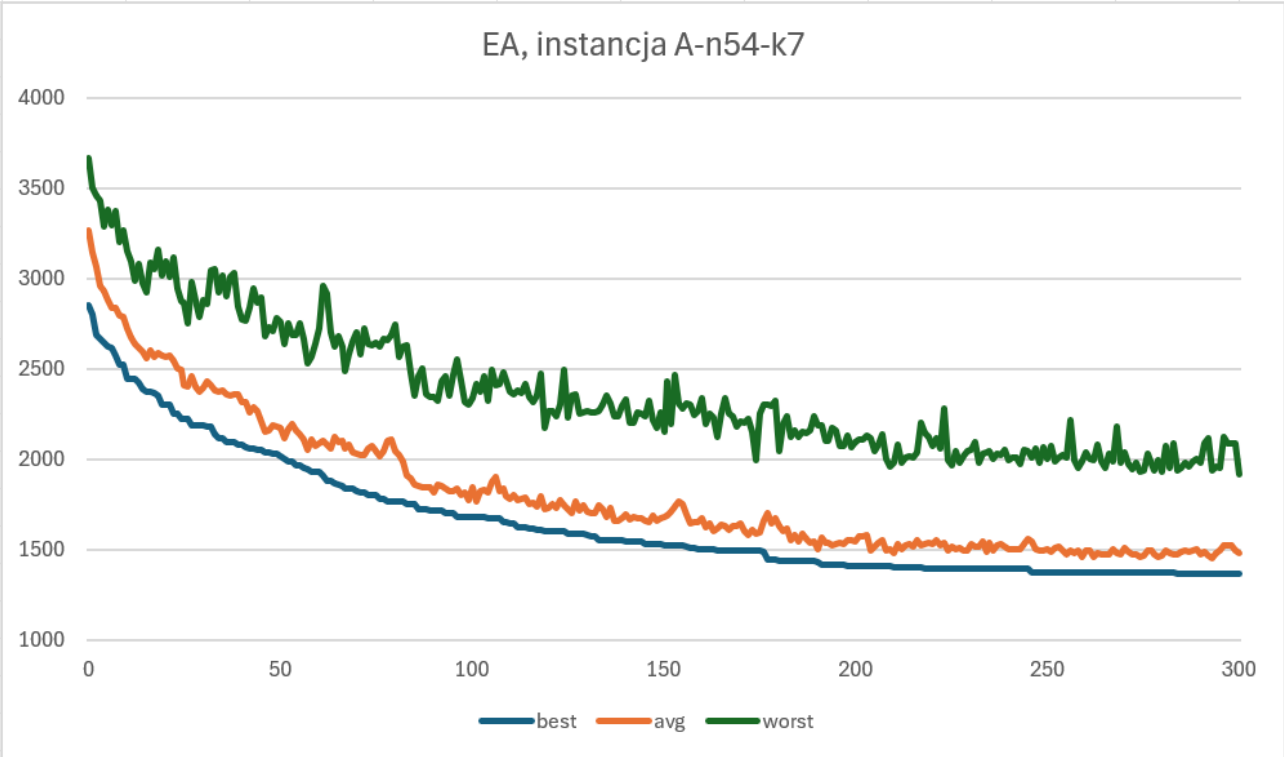
instancja	Opt.	Alg.losowy [30k]				Alg.zachłanny [n]				Alg.Ewolucyjny [10x]				TS [10x]		
	wynik	**best**	worst	avg	std	**best**	worst	avg	std	**best**	worst	avg	std	**best**	worst	avg
A-n54-k7	1167	**2468,09**	3900,81	3293,81	172,875	**1360,48**	1621,82	1524,48	62,7286	**1292,89**	1522,4	1407,66	90,6679	**1274,67**	1527,44	1444,91
A-n60-k9	1354	**2973,1**	4354,85	3698,74	179,248	**1686,52**	1972,17	1852,75	68,7713	**1579,54**	1756,15	1667,35	65,8751	**1601,63**	1809,56	1696,55

W algorytmie ewolucyjnym jedyną zmianą względem wcześniejszych eksperymentów była liczba generacji — zwiększona ze 100 do 300; wszystkie pozostałe parametry, krzyżowanie OX, mutacja inwersją, pozostały bez zmian. Dla rzetelności porównania podniesiono też liczbę iteracji algorytmu losowego do 30 000, natomiast pozostałe metody działały jak w rozdziale 3.

Mimo podniesienia liczby iteracji do 30 000, algorytm losowy nie odnotował poprawy jakości rozwiązań — uzyskane wartości best i avg wciąż pozostają wyraźnie oddalone od optimum. Oznacza to, że samo zwiększanie liczby losowych prób nie kompensuje braku ukierunkowanego przeszukiwania.

Zwiększenie liczby generacji w EA przełożyło się na wyraźną poprawę jakości: algorytm ewolucyjny osiąga wyniki bliskie optimum i systematycznie lepsze od Greedy — zarówno w best, jak i avg. Niewielkie odchylenia standardowe ($std \approx 66$–91) potwierdzają stabilność przebiegu i skuteczną eksploatację przestrzeni rozwiązań.

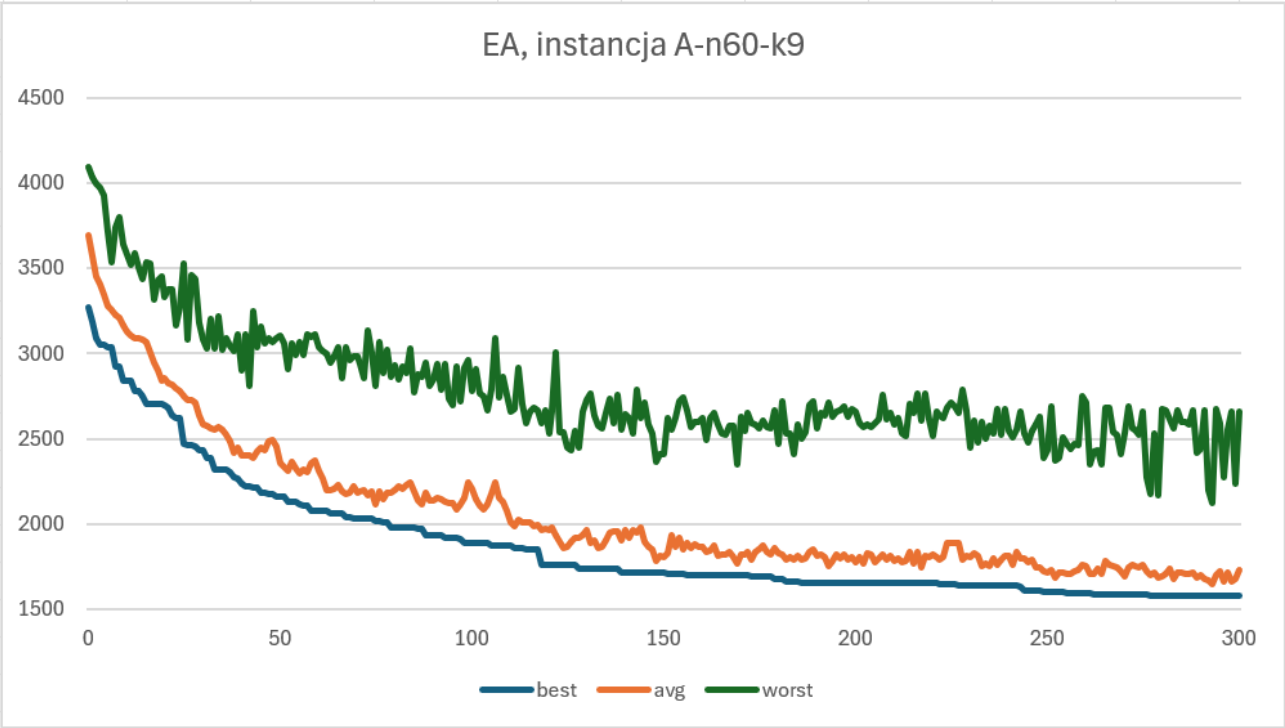
Tabu Search uzyskał dla A-n54-k7 najlepsze best (1274,67), podczas gdy EA wygrał na A-n60-k9 (1579,54 vs 1601,63). Co istotne, EA ma lepsze średnie wartości avg od TS w obu przypadkach (1407,66 vs 1444,91 dla A-n54-k7 oraz 1667,35 vs 1696,55 dla A-n60-k9), co świadczy o bardziej równomiernej jakości populacji. Podsumowując: zwiększenie liczby generacji z 100 do 300 pozwoliło EA zniwelować dystans do optimum i zdominować Greedy, a także dorównać (lub przewyższyć średnio) Tabu Search w najtrudniejszych instancjach. W najtrudniejszych instancjach głównym ograniczeniem nie był typ mutacji ani niepoprawnie dostrojone parametry, lecz zbyt mały budżet ewolucji (liczba cykli selekcja–krzyżowanie–mutacja). Dla trudnych instancji ważniejsze od agresywnej zmiany operatorów okazało się zwiększenie liczby generacji, ewentualnie wsparte prostą mutacją i umiarkowaną presją selekcyjną.



Rysunek 15: Wykres algorytmu ewolucyjnego dla instancji A-n54-k7.

EA – najlepsze rozwiązanie dla A-n54-k7

- **Route 1:** 19, 6, 40, 50, 37, 22, 34, 10, 4, 16
- **Route 2:** 31, 26, 48, 42, 52, 25, 43, 47, 35, 53
- **Route 3:** 28, 3, 49, 41, 32, 9, 20, 14
- **Route 4:** 12, 54, 45, 39, 11
- **Route 5:** 17, 36, 24, 21, 8, 51, 29, 5
- **Route 6:** 7, 23, 38, 13, 15, 33
- **Route 7:** 44, 18, 27, 46, 2, 30



Rysunek 16: Wykres algorytmu ewolucyjnego dla instancji A-n60-k9.

EA – najlepsze rozwiązanie dla A-n60-k9

- **Route 1:** 34, 8, 58, 18, 28, 14, 19
- **Route 2:** 2, 49, 37, 3, 7
- **Route 3:** 26, 47, 59, 6, 43, 44, 56, 15
- **Route 4:** 41, 32, 50, 31, 54, 5, 4, 21, 17
- **Route 5:** 12, 22, 29, 45, 25, 35
- **Route 6:** 40, 33, 10, 52, 13, 57, 51, 36
- **Route 7:** 48, 24, 16, 27, 38, 30, 20
- **Route 8:** 42, 39, 60, 9, 53
- **Route 9:** 46, 55, 11, 23

4.9 Badanie wpływu wartości dla różnych parametrów algorytmu ewolucyjnego

Do badań została wykorzystana instancja A-n39-k5 oraz parametry początkowe, takie jak:

- Rozmiar populacji: 100
- Ilość generacji: 100
- Prawdopodobieństwo krzyżowania: 0.7
- Rodzaj krzyżowania: OX
- Prawdopodobieństwo mutacji: 0.4
- Rodzaj mutacji: inwersja
- Rozmiar turnieju: 5
- Rozmiar elity: 1

Tabela 3: Wpływ wielkości populacji na wyniki algorytmu

Wielkość populacji	Best	Worst	Avg
30	1048	1247	1149
100	976	1169	1060
300	941	1020	989

W kolejnych testach zmieniano po jednym parametrze, zgodnie z przedstawionymi poniżej tabelami.

Wpływ wielkości populacji

Najlepsze wyniki zostały osiągnięte przy największej populacji równej 300. Jest to potwierdzenie, że większa różnorodność genetyczna sprzyja znajdowaniu lepszych rozwiązań. Najgorsze rezultaty uzyskano przy najmniejszej populacji 30, gdzie ograniczona pula osobników utrudnia eksplorację przestrzeni rozwiązań i sprzyja przedwczesnej zbieżności. Średnie wartości również poprawiają się wraz ze wzrostem populacji, co wskazuje, że większa liczba osobników zwiększa stabilność działania algorytmu i umożliwia lepsze równoważenie eksploracji i eksploatacji.

Wpływ liczby pokoleń

Tabela 4: Wpływ liczby pokoleń na wyniki algorytmu

Liczba pokoleń	Best	Worst	Avg
50	1062	1207	1123
150	898	1102	997
500	914	999	938

Najlepszy wynik został uzyskany przy liczbie pokoleń równej 150, co sugeruje, że umiarkowana liczba iteracji pozwala algorytmowi na efektywną poprawę rozwiązań bez nadmiernego czasu obliczeń. Choć dla 500 pokoleń wartości średnie są nieco lepsze niż dla 150, czas obliczeń jest niepotrzebnie wydłużany, przy czym najlepszy wynik nie jest lepszy niż ten uzyskany przy 150 pokoleniach. Najgorsze wyniki pojawiły się przy 50 pokoleniach, gdzie algorytm nie miał wystarczająco dużo czasu na ewolucję populacji. Potwierdza to, że zbyt mała liczba pokoleń ogranicza zdolność do uzyskania rozwiązań bliskich optimum.

Wpływ prawdopodobieństwa krzyżowania

Tabela 5: Wpływ prawdopodobieństwa krzyżowania na wyniki algorytmu

Prawdopodobieństwo krzyżowania	Best	Worst	Avg
0,1	927	1144	1043
0,5	1003	1104	1069
0,9	910	1164	1017

Wyniki pokazują, że najlepsze efekty osiągnięto przy prawdopodobieństwie krzyżowania 0,9, gdzie uzyskano najniższy koszt best - 910 oraz najniższą średnią wartość 1017. Sugeruje to, że wysokie prawdopodobieństwo krzyżowania sprzyja intensywnej wymianie informacji pomiędzy osobnikami i prowadzi do bardziej optymalnych rozwiązań. Dla wartości 0,1 i 0,5 wyniki są gorsze — szczególnie przy 0,5, gdzie średni koszt 1069 jest najwyższy. Może to oznaczać, że zbyt umiarkowane krzyżowanie nie zapewnia wystarczającej różnorodności populacji lub sprzyja utknięciu w lokalnych minimach.

Wpływ prawdopodobieństwa mutacji

Tabela 6: Wpływ prawdopodobieństwa mutacji na wyniki algorytmu

Prawdopodobieństwo mutacji	Best	Worst	Avg
0,1	1063	1223	1132
0,5	972	1116	1022
0,9	1068	1258	1146

Najlepsze rezultaty zanotowano dla wartości mutacji 0,5, co sugeruje, że umiarkowana intensywność mutacji skutecznie utrzymuje różnorodność genetyczną i zapobiega stagnacji. Najgorsze wyniki pojawiły się przy bardzo wysokim poziomie mutacji 0,9, który wprowadza zbyt duży chaos i rozbija dobre rozwiązania. Z kolei niska mutacja 0,1 daje bardziej stabilne, lecz mniej optymalne wyniki, co wskazuje na nadmierną eksploatację.

Wyniki potwierdzają, że mutacja musi być dobrana z umiarem — zbyt mała ogranicza eksplorację, a zbyt duża niszczy strukturę populacji.

Wpływ rozmiaru turnieju

Tabela 7: Wpływ rozmiaru turnieju na wyniki algorytmu

Rozmiar turnieju	Best	Worst	Avg
3	1040	1252	1139
5	904	1084	1010
10	938	1113	1007

Najlepszy wynik został osiągnięty przy rozmiarze turnieju równym 5, co wskazuje na optymalne ciśnienie selekcyjne — wystarczające do promowania lepszych osobników, ale nie na tyle silne, by utracić różnorodność. Przy rozmiarze 3 wyniki są wyraźnie gorsze, co sugeruje zbyt agresywną selekcję prowadzącą do szybkiej zbieżności i ryzyka utknięcia w lokalnym minimum. Z kolei przy wartości 10 wyniki są stabilne, ale nieco słabsze w stosunku do turnieju 5, co może być skutkiem zbyt małego nacisku selekcyjnego. Optymalna siła selekcji jest więc kluczowa dla równowagi między eksploracją a eksploatacją.

Wpływ rodzaju krzyżowania
Zastosowane parametry:

- Rozmiar populacji: 100
- Ilość generacji: 100
- Prawdopodobieństwo krzyżowania: 0.7
- Prawdopodobieństwo mutacji: 0.4
- Rodzaj mutacji: inwersja
- Rozmiar turnieju: 5
- Rozmiar elity: 1

Tabela 8: Wpływ rodzaju krzyżowania na wyniki algorytmu

Rodzaj krzyżowania	Best	Worst	Avg
OX	963	1187	1035
PMX	915	1098	1006

W przedstawionych eksperymentach operator PMX uzyskał lepsze wyniki od OX zarówno pod względem najlepszego rozwiązania (915 vs 963), jak i średniej jakości populacji (1006 vs 1035). Sugeruje to, że w badanych ustawieniach PMX skuteczniej zachowuje i przenosi spójne fragmenty rozwiązań (mapowanie genów ogranicza konflikty i duplikacje), co sprzyja intensyfikacji poszukiwań wokół obiecujących schematów. Z kolei OX, choć dobrze utrzymuje względny porządek elementów rodzica, może słabiej odbudowywać pewne kluczowe bloki, co przekłada się na nieco gorsze statystyki. Przy zadanych parametrach PMX zapewnia korzystniejszą równowagę między eksploatacją a eksploracją, prowadząc do niższych kosztów końcowych i stabilniejszych przebiegów.

Wpływ rodzaju mutacji
Zastosowane parametry dla mutacji typu inwersja:

- Rozmiar populacji: 100
- Ilość generacji: 100
- Prawdopodobieństwo krzyżowania: 0.7
- Rodzaj krzyżowania: OX
- Prawdopodobieństwo mutacji: 0.4
- Rozmiar turnieju: 5

- Rozmiar elity: 1

Zastosowane parametry dla mutacji typu swap:

- Rozmiar populacji: 100
- Ilość generacji: 100
- Prawdopodobieństwo krzyżowania: 0.7
- Rodzaj krzyżowania: OX
- Prawdopodobieństwo mutacji: 0.01
- Rozmiar turnieju: 5
- Rozmiar elity: 1

Tabela 9: Wpływ rodzaju mutacji na wyniki algorytmu

Rodzaj mutacji	**Best**	**Worst**	**Avg**
Inwersja (per osobnik)	963	1187	1035
Swap (per gen)	977	1132	1077

Lepsze wyniki uzyskano dla inwersji: niższy koszt najlepszego rozwiązania (963 vs 977) oraz lepsza średnia populacji (1035 vs 1077). Wynik ten jest spójny z charakterem obu operatorów i użytą skalą parametrów. Inwersja działa „blokowo": pojedyncza operacja (wykonywana z prawdopodobieństwem $P_m = 0{,}4$ na osobnika) potrafi odwrócić dłuższy fragment permutacji, skutecznie przestawiając duże segmenty trasy i intensyfikując przeszukiwanie w otoczeniu dobrych schematów. Z kolei swap (per gen), mimo znacznie niższego $P_m = 0{,}01$, wykonuje wiele drobnych perturbacji w pojedynczym osobniku, co sprzyja utrzymaniu różnorodności, ale może rozpraszać eksploatację wzorców przekazywanych przez OX i spowalniać zejście do lepszego optimum. W podanej konfiguracji inwersja zapewnia korzystniejszy kompromis eksploracja/eksploatacja. Natomiast w przypadku swapa, warto rozważyć drobną kalibrację P_m (np. w zakresie 0,005–0,009) i/lub minimalne zwiększenie P_x (0,85–0,90), aby uzyskać wynik bliższy optimum.

5 Podsumowanie

Analiza uzyskanych wyników jednoznacznie wskazuje na znaczące różnice w skuteczności pomiędzy zastosowanymi metodami optymalizacyjnymi. Algorytm losowy, mimo dużej liczby iteracji (10 000 generowanych rozwiązań), uzyskał zdecydowanie najgorsze wyniki we wszystkich instancjach. Zarówno wartości średnie, jak i skrajne (best/worst), znacząco odbiegały od optymalnych, co potwierdza brak mechanizmów eksploracyjno-eksploatacyjnych zdolnych do stopniowej poprawy rozwiązań. Wysokie wartości odchylenia standardowego świadczą dodatkowo o dużej niestabilności i przypadkowości generowanych tras.

Zastosowany algorytm zachłanny, korzystający z nearest neighbor, pozwolił na znaczącą poprawę jakości tras w porównaniu z podejściem losowym. Wyniki średnie w każdej instancji były zdecydowanie bliższe optimum, co dowodzi skuteczności heurystyk deterministycznych w szybkiej generacji poprawnych rozwiązań początkowych. Niemniej jednak, brak mechanizmów ulepszających spowodował zatrzymanie się algorytmu w lokalnych minimach, co widoczne jest w szerokich różnicach pomiędzy wartościami best i worst w instancjach o większej złożoności.

Algorytm ewolucyjny wykazał najwyższą efektywność w większości przypadków w porównaniu do poprzednich metod konstrukcyjnych. Poprzez zastosowanie operatorów krzyżowania i mutacji, a także selekcji turniejowej, był on zdolny do systematycznego ulepszania populacji rozwiązań. Wartości best były konsekwentnie zbliżone do wyników optymalnych, a wartości średnie utrzymywały się na stabilnym poziomie. Niewielkie odchylenia standardowe wskazują, że pomimo natury niedeterministycznej algorytm osiągał powtarzalne rezultaty, co jest kluczowe w kontekście niezawodności metod heurystycznych.

Najlepsze wyniki globalne zaobserwowano w przypadku algorytmu Tabu Search (TS). Metoda ta, dzięki pamięci tabu oraz mechanizmom dopuszczania ruchów chwilowo pogarszających wynik, była najbardziej odporna na przedwczesną zbieżność. w większości instancji wartości best osiągane przez TS były najbliższe optimum, a wartości avg przewyższały jakością zarówno algorytm zachłanny, jak i ewolucyjny. Jednocześnie relatywnie

niskie wartości worst świadczą o dobrej stabilności działania. w porównaniu do algorytmu ewolucyjnego TS wykazał większą zdolność eksploracji przestrzeni rozwiązań przy jednoczesnym zachowaniu kontroli nad jakością generowanych tras.

Podsumowując, metody metaheurystyczne (EA i TS) zdecydowanie przewyższają heurystyki proste (losowa, zachłanna) zarówno pod względem jakości uzyskanych rozwiązań, jak i stabilności działania. Tabu Search okazał się najskuteczniejszy w osiąganiu wyników najbliższych optimum, natomiast algorytm ewolucyjny zapewniał najlepszy kompromis pomiędzy jakością, stabilnością oraz kosztami obliczeniowymi. Ponadto w przypadku algorytmu ewolucyjnego istnieje wciąż duży potencjał poprawy związany z dostrajaniem parametrów oraz zwiększaniem liczby pokoleń, czy populacji. Zastosowanie metod takich jak algorytm losowy, czy zachłanny może być rekomendowane jedynie do generowania rozwiązań początkowych, w celu zwiększenia efektywności metaheurystyk lub jako komponent w algorytmach hybrydowych.

Podsumowując:

- Algorytm losowy pełni rolę jedynie porównawczą i nie nadaje się do realnego rozwiązywania problemów cVRP.
- Algorytm zachłanny zapewnia szybkie i powtarzalne wyniki, lecz brakuje mu globalnej eksploracji.
- Algorytm ewolucyjny charakteryzuje się najlepszym kompromisem między jakością a stabilnością wyników oraz dużym potencjałem dostrajania parametrów.
- Tabu Search stanowi silną alternatywę dla EA, szczególnie w trudnych instancjach, oferując wysoką jakość rozwiązań dzięki mechanizmowi pamięci.

Dzięki zestawieniu wyników można jednoznacznie określić przewagę metod metaheurystycznych nad metodami prostymi, a w szczególności potwierdzić skuteczność algorytmu ewolucyjnego jako głównego rozwiązania problemu marszrutyzacji pojazdów.